

Sistema de Control

Hospital

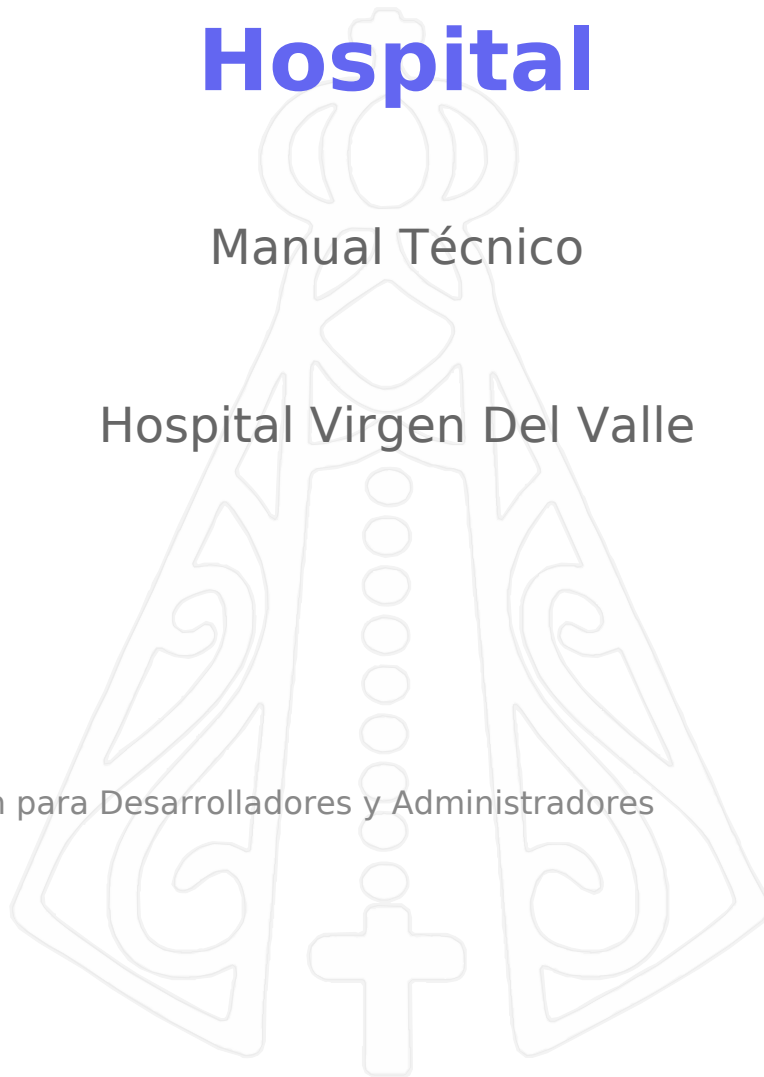
Manual Técnico

Hospital Virgen Del Valle

Versión 1.0

Enero 2026

Documentación para Desarrolladores y Administradores



Índice

1. Arquitectura del Sistema
2. Base de Datos
3. Rutas y Endpoints
4. Modelos y Relaciones
5. Controladores
6. Vistas y Frontend
7. Generación de PDFs y QR
8. Instalación y Despliegue

1. Arquitectura del Sistema

1.1 Stack Tecnológico

Componente	Tecnología	Versión
Framework Backend	Laravel	11.x
Lenguaje	PHP	8.2+
Base de Datos	MySQL	8.0+
Framework CSS	Tailwind CSS	4.0
Motor de Plantillas	Blade	Laravel 11
Generación PDF	DomPDF	2.x
Generación QR	SimpleSoftwareIO/simple-qrcode	4.x

1.2 Patrón de Arquitectura

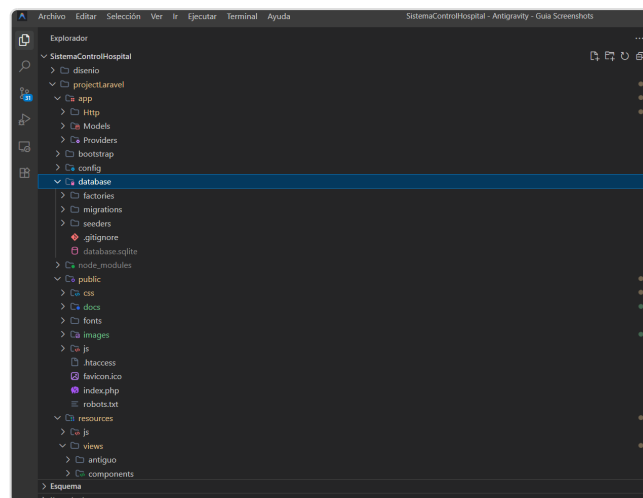
El sistema implementa el patrón MVC (Model-View-Controller) de Laravel:

- **Modelos:** Representan las entidades de la base de datos (BN, AireAcondicionado, Mantenimiento, etc.)
- **Vistas:** Plantillas Blade que renderizan la interfaz de usuario
- **Controladores:** Lógica de negocio que procesa las peticiones

1.3 Estructura de Directorios

```
projectLaravel/  
??? app/  
?   ??? Http/
```

```
? ? ??? Controllers/
? ?     ??? BNController.php
? ?     ??? AirAcondController.php
? ?     ??? MaintenanceController.php
? ?     ??? ReportesController.php
? ?     ??? DashboardController.php
? ?     ??? ManualController.php
? ??? Models/
?     ??? BN.php
?     ??? AireAcondicionado.php
?     ??? Mantenimiento.php
?     ??? ReportesBN.php
?     ??? Area.php
?     ??? Categoria.php
??? database/
? ??? migrations/
??? public/
? ??? docs/
? ??? images/
??? resources/
? ??? views/
?     ??? layouts/
?     ??? modals/
?     ??? manuals/
?     ??? *.blade.php
??? routes/
??? web.php
```



1.4 Flujo de Datos

El flujo típico de una petición es:

1. Usuario realiza una acción en la interfaz
2. La ruta en `web.php` captura la petición

3. El controlador correspondiente procesa la lógica
4. El modelo interactúa con la base de datos si es necesario
5. El controlador retorna una vista o respuesta JSON
6. La vista Blade renderiza el HTML final

2. Base de Datos

2.1 Esquema de Tablas

EstructuraSQLBuscarGenerar una consultaExportarImportarOperacionesPrivilegiosRutinasEventosMás

Filtros

Que contengan la palabra:

Tabla	Acción	Filas	Tipo	Cotejamiento	Tamaño	Residuo a depurar
☐ aires acondicionados	Examinar Estructura Buscar Insertar Vaciar Eliminar	15	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐ areas	Examinar Estructura Buscar Insertar Vaciar Eliminar	5	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
☐ bienes_nacionales	Examinar Estructura Buscar Insertar Vaciar Eliminar	25	InnoDB	utf8mb4_unicode_ci	64.0 KB	-
☐ categorias	Examinar Estructura Buscar Insertar Vaciar Eliminar	4	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
☐ materiales_faltantes	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐ migrations	Examinar Estructura Buscar Insertar Vaciar Eliminar	3	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
☐ password_reset_tokens	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	16.0 KB	-
☐ reportes_aa	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐ reportes_bn	Examinar Estructura Buscar Insertar Vaciar Eliminar	7	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
☐ sessions	Examinar Estructura Buscar Insertar Vaciar Eliminar	3	InnoDB	utf8mb4_unicode_ci	48.0 KB	-
☐ users	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB	-
11 tablas	Número de filas	62	InnoDB	utf8mb4_general_ci	336.0 KB	0 B

☐ Seleccionar todo

Para los elementos que están marcados:

Imprimir Diccionario de datos

Crear nueva tabla

2.1.1 Tabla: bns (Bienes Nacionales)

Campo	Tipo	Descripción
id	BIGINT	Clave primaria
numero_bien	VARCHAR(255)	Número único del bien nacional
descripcion	TEXT	Descripción del bien
area_id	BIGINT	FK a tabla areas
categoria_id	BIGINT	FK a tabla categorias
estado	VARCHAR(50)	Estado actual del bien
observaciones	TEXT	Observaciones adicionales

created_at	TIMESTAMP	Fecha de creación
updated_at	TIMESTAMP	Fecha de actualización

2.1.2 Tabla: aires_acondicionados

Campo	Tipo	Descripción
id	BIGINT	Clave primaria
nombre	VARCHAR(255)	Nombre del equipo
ubicacion	VARCHAR(255)	Ubicación/Área
capacidad	DECIMAL(5,2)	Capacidad en toneladas
estado	VARCHAR(50)	Estado operativo
marca	VARCHAR(100)	Marca del equipo
modelo	VARCHAR(100)	Modelo del equipo
numero_serie	VARCHAR(100)	Número de serie
tipo_refrigerante	VARCHAR(50)	Tipo de refrigerante
voltaje	VARCHAR(20)	Voltaje de operación
amperaje	VARCHAR(20)	Amperaje
fecha_instalacion	DATE	Fecha de instalación
horas_uso	INTEGER	Horas de uso acumuladas
created_at	TIMESTAMP	Fecha de creación
updated_at	TIMESTAMP	Fecha de actualización

2.1.3 Tabla: mantenimientos

Campo	Tipo	Descripción
id	BIGINT	Clave primaria
aire_acondicionado_id	BIGINT	FK a aires_acondicionados
tipo	VARCHAR(50)	Preventivo/Correctivo
fecha	DATE	Fecha del mantenimiento
tecnico	VARCHAR(255)	Técnico responsable
descripcion	TEXT	Descripción de trabajos
materiales	JSON	Materiales utilizados
observaciones	TEXT	Observaciones
created_at	TIMESTAMP	Fecha de creación
updated_at	TIMESTAMP	Fecha de actualización

2.1.4 Tabla: reportes_bns

Campo	Tipo	Descripción
id	BIGINT	Clave primaria
bn_id	BIGINT	FK a bns
tipo_actividad	VARCHAR(100)	Tipo de reporte
fecha	DATE	Fecha del reporte
descripcion	TEXT	Descripción del reporte
responsable	VARCHAR(255)	Responsable del reporte
created_at	TIMESTAMP	Fecha de creación

updated_at	TIMESTAMP	Fecha de actualización
------------	-----------	------------------------

2.2 Relaciones

- **BN → Area:** Muchos a Uno (belongsTo)
- **BN → Categoria:** Muchos a Uno (belongsTo)
- **BN → ReportesBN:** Uno a Muchos (hasMany)
- **AireAcondicionado → Mantenimiento:** Uno a Muchos (hasMany)

2.3 Migraciones

Las migraciones se encuentran en `database/migrations/`. Para ejecutarlas:

```
php artisan migrate
```

Para revertir migraciones:

```
php artisan migrate:rollback
```

3. Rutas y Endpoints

3.1 Rutas Principales

Definidas en `routes/web.php` :

Dashboard

```
Route::get('/inicio', DashboardController::class)->name('inicio');
```

Bienes Nacionales

```
Route::prefix('bienes-nacionales')->group(function () {  
    Route::get('/', [BNController::class, 'index'])->name('bienes-nacionales.index');  
    Route::post('/', [BNController::class, 'store'])->name('bienes-nacionales.store');  
    Route::put('/{id}', [BNController::class, 'update'])->name('bienes-nacionales.update');  
    Route::delete('/{id}', [BNController::class, 'destroy'])->name('bienes-nacionales.destroy');  
    Route::get('/history/{id}', [BNController::class, 'history'])->name('bienes-nacionales.history');  
    Route::get('/history/{id}/pdf', [BNController::class, 'downloadHistoryPDF'])->name('bienes-nacionales.history.pdf');  
    Route::get('/{id}/pdf', [BNController::class, 'downloadPDF'])->name('bienes-nacionales.pdf');  
    Route::get('/{id}/qr', [BNController::class, 'generateQR'])->name('bienes-nacionales.qr');  
});
```

Aires Acondicionados

```
Route::get('/aires-acondicionados', [AirAcondController::class, 'index'])->name('aires-acondicionados.index');  
Route::post('/aires-acondicionados', [AirAcondController::class, 'store'])->name('aires-acondicionados.store');  
Route::get('/aires-acondicionados/{id}/edit', [AirAcondController::class, 'edit'])->name('aires-acondicionados.edit');  
Route::put('/aires-acondicionados/{id}', [AirAcondController::class, 'update'])->name('aires-acondicionados.update');  
Route::get('/aires-acondicionados/{id}', [AirAcondController::class, 'show'])->
```

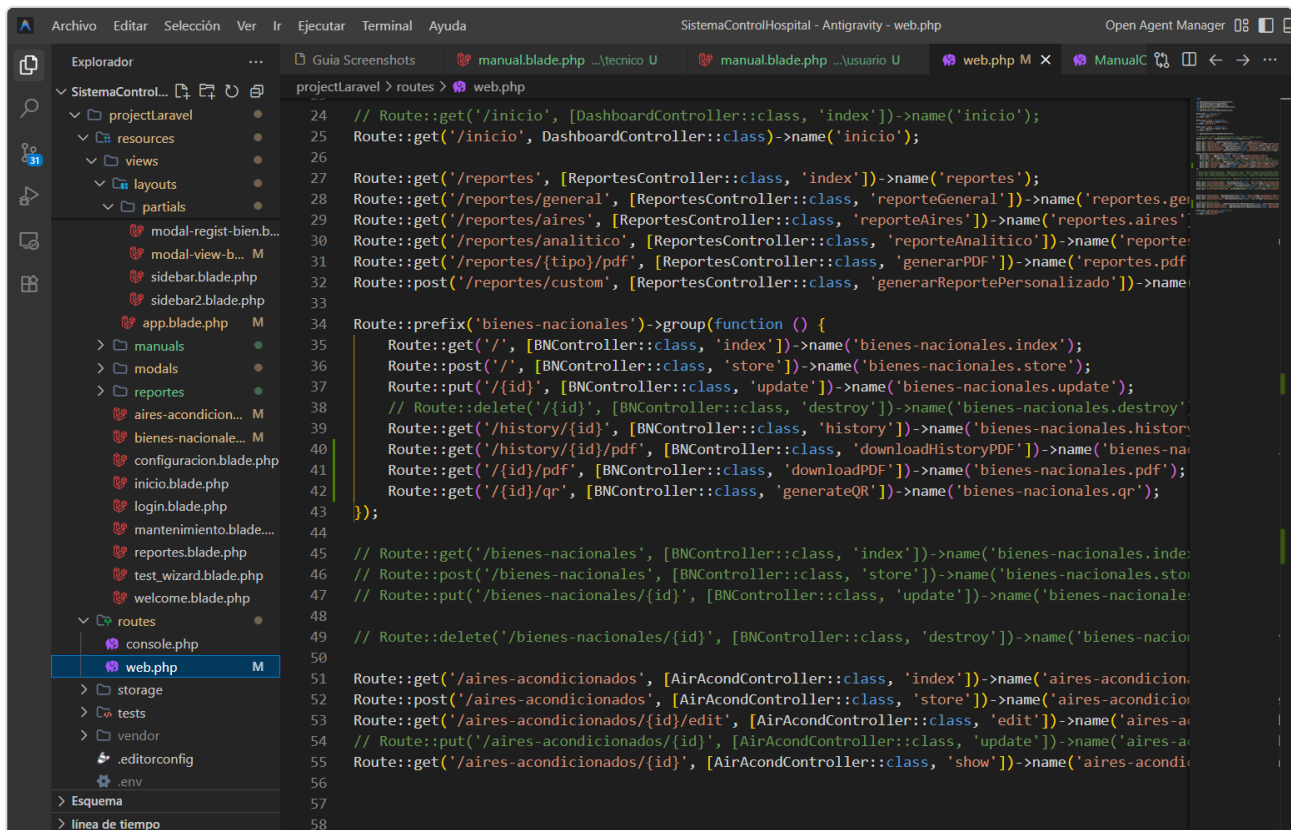
```
>name('aires-acondicionados.show');
```

Mantenimientos

```
Route::get('/mantenimiento', [MaintenanceController::class, 'index'])->>name('mantenimiento.index');
Route::post('/mantenimiento', [MaintenanceController::class, 'store'])->>name('mantenimiento.store');
Route::get('/mantenimiento/{id}', [MaintenanceController::class, 'show'])->>name('mantenimiento.show');
```

Reportes

```
Route::get('/reportes', [ReportesController::class, 'index'])->>name('reportes');
Route::get('/reportes/general', [ReportesController::class, 'reporteGeneral'])->>name('reportes.general');
Route::get('/reportes/aires', [ReportesController::class, 'reporteAires'])->>name('reportes.aires');
Route::get('/reportes/analitico', [ReportesController::class, 'reporteAnalitico'])->>name('reportes.analitico');
Route::get('/reportes/{tipo}/pdf', [ReportesController::class, 'generarPDF'])->>name('reportes.pdf');
```



3.2 Middleware

Las rutas están protegidas por middleware de autenticación (si está configurado).

4. Modelos y Relaciones

4.1 Modelo BN

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

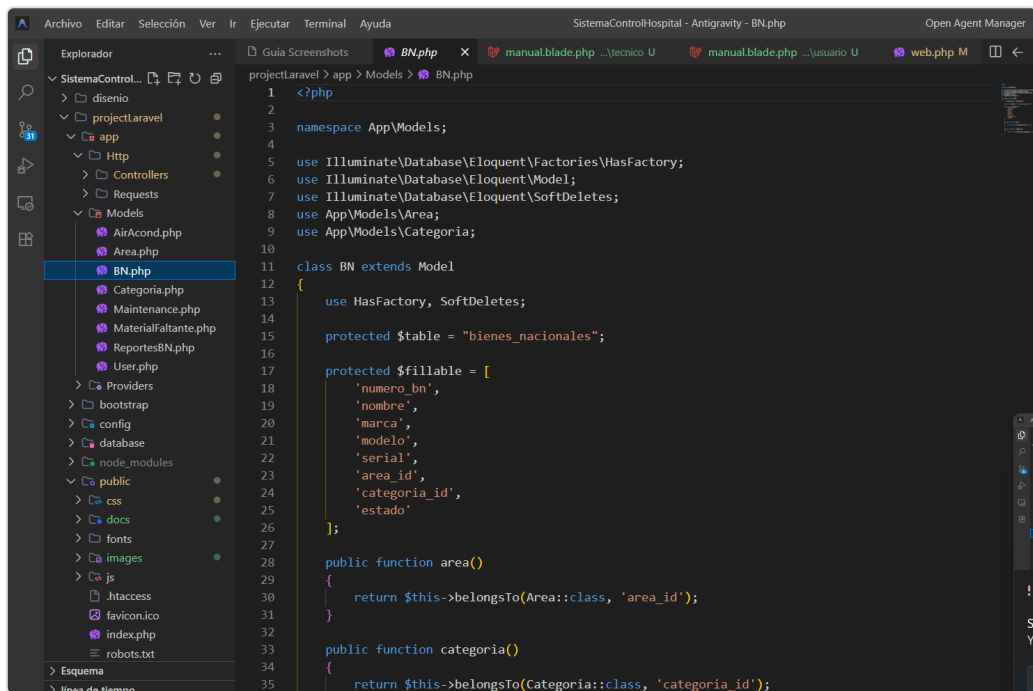
class BN extends Model
{
    protected $table = 'bns';

    protected $fillable = [
        'numero_bien',
        'descripcion',
        'area_id',
        'categoria_id',
        'estado',
        'observaciones'
    ];

    // Relaciones
    public function area()
    {
        return $this->belongsTo(Area::class);
    }

    public function categoria()
    {
        return $this->belongsTo(Categoria::class);
    }

    public function reportes()
    {
        return $this->hasMany(ReportesBN::class);
    }
}
```



4.2 Modelo AireAcondicionado

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class AireAcondicionado extends Model
{
    protected $table = 'aires_acondicionados';

    protected $fillable = [
        'nombre',
        'ubicacion',
        'capacidad',
        'estado',
        'marca',
        'modelo',
        'numero_serie',
        'tipo_refrigerante',
        'voltaje',
        'amperaje',
        'fecha_instalacion',
        'horas_uso'
    ];

    protected $casts = [
        'fecha_instalacion' => 'date',
        'capacidad' => 'decimal:2'
    ];
}
```

```
// Relaciones
public function mantenimientos()
{
    return $this->hasMany(Mantenimiento::class);
}
}
```

4.3 Modelo Mantenimiento

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Mantenimiento extends Model
{
    protected $table = 'mantenimientos';

    protected $fillable = [
        'aire_acondicionado_id',
        'tipo',
        'fecha',
        'tecnico',
        'descripcion',
        'materiales',
        'observaciones'
    ];

    protected $casts = [
        'fecha' => 'date',
        'materiales' => 'array'
    ];

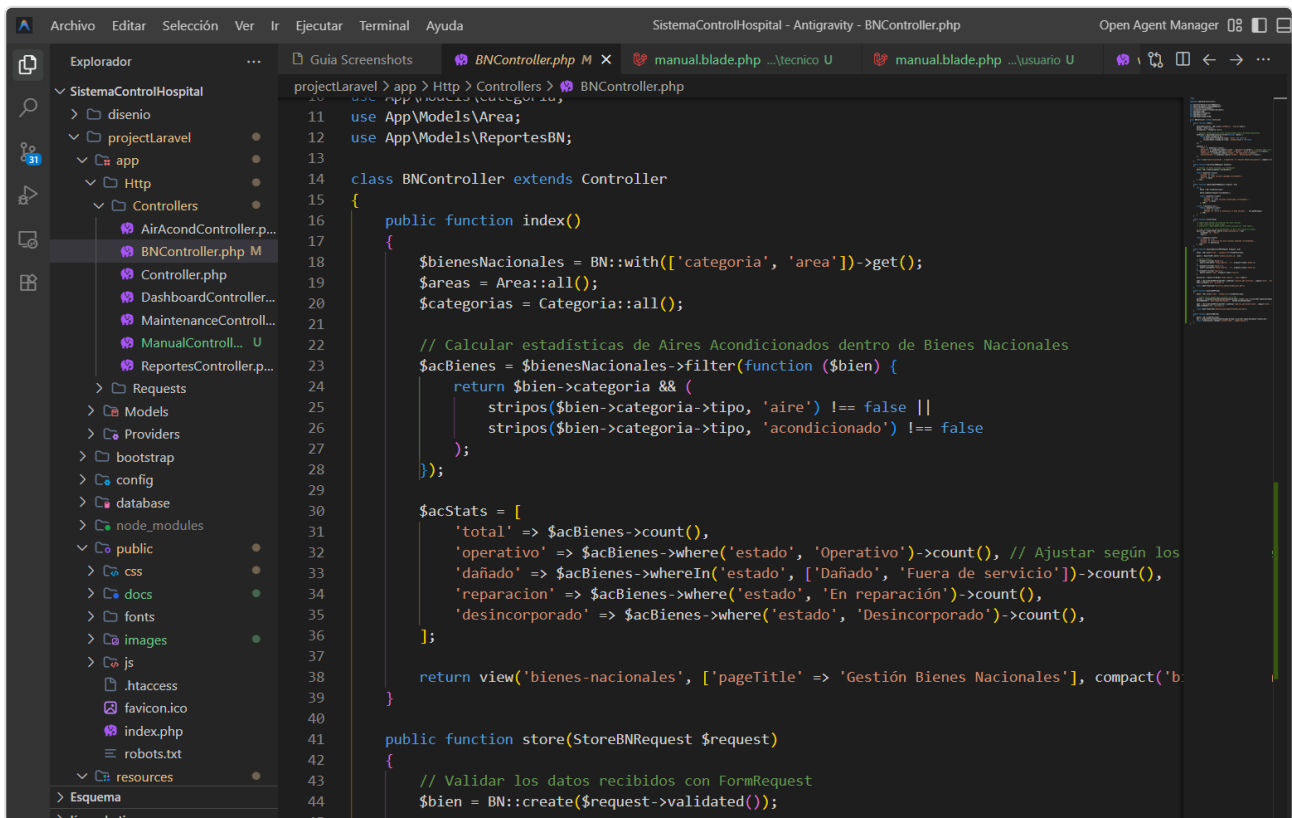
    // Relaciones
    public function aireAcondicionado()
    {
        return $this->belongsTo(AireAcondicionado::class);
    }
}
```

5. Controladores

5.1 BNController

Gestiona todas las operaciones CRUD de bienes nacionales:

- `index()` - Lista todos los bienes
- `store(Request $request)` - Crea un nuevo bien
- `update(Request $request, $id)` - Actualiza un bien
- `destroy($id)` - Elimina un bien
- `history($id)` - Obtiene el historial de reportes
- `downloadPDF($id)` - Genera PDF del bien
- `downloadHistoryPDF($id)` - Genera PDF del historial
- `generateQR($id)` - Genera código QR



5.2 AirAcondController

Gestiona equipos de aire acondicionado:

- `index()` - Lista todos los equipos AC
- `store(Request $request)` - Registra nuevo equipo
- `show($id)` - Muestra detalles del equipo
- `edit($id)` - Prepara datos para edición
- `update(Request $request, $id)` - Actualiza equipo

5.3 MaintenanceController

Gestiona mantenimientos de equipos AC:

- `index()` - Lista mantenimientos
- `store(Request $request)` - Registra mantenimiento
- `show($id)` - Muestra detalles del mantenimiento

5.4 ReportesController

Genera reportes del sistema:

- `index()` - Vista principal de reportes
- `reporteGeneral()` - Reporte general
- `reporteAires()` - Reporte de AC
- `reporteAnalitico()` - Reporte analítico
- `generarPDF($tipo)` - Genera PDF del reporte

5.5 DashboardController

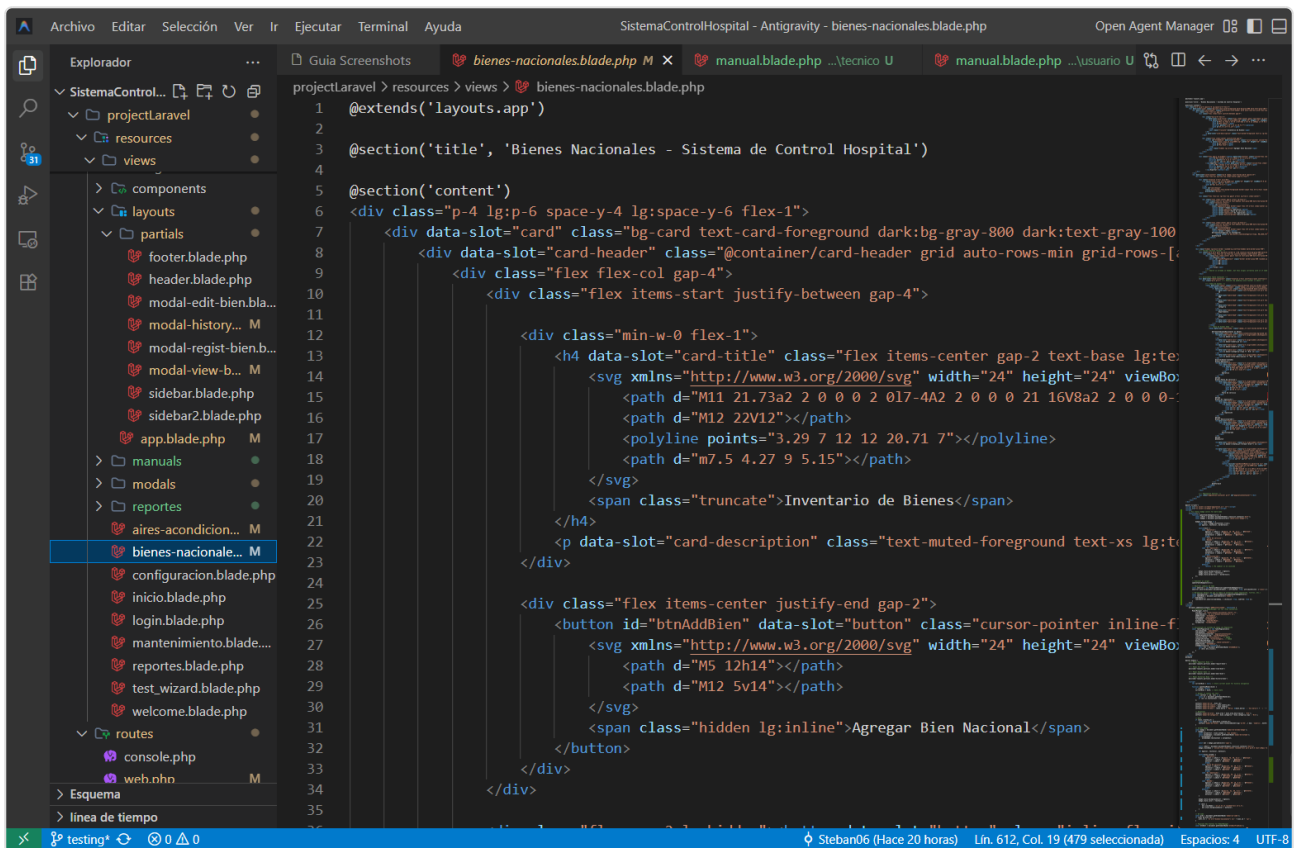
Muestra estadísticas del sistema:

- `__invoke()` - Retorna vista del dashboard con estadísticas

6. Vistas y Frontend

6.1 Estructura de Vistas

```
resources/views/  
??? layouts/  
?   ??? app.blade.php (Layout principal)  
?   ??? partials/  
?       ??? header.blade.php  
?       ??? sidebar.blade.php  
?       ??? footer.blade.php  
??? modals/  
?   ??? add_bien.blade.php  
?   ??? view_bien.blade.php  
?   ??? edit_bien.blade.php  
?   ??? modal-history-bien.blade.php  
?   ??? add_ac.blade.php  
?   ??? view_ac.blade.php  
?   ??? edit_ac.blade.php  
?   ??? history_ac.blade.php  
?   ??? register_maintenance.blade.php  
?   ??? add_missing_material.blade.php  
??? bienes-nacionales.blade.php  
??? aires-acondicionados.blade.php  
??? reportes.blade.php  
??? configuracion.blade.php  
??? welcome.blade.php
```



6.2 Tailwind CSS v4

El sistema utiliza Tailwind CSS v4 para el diseño. Características principales:

- Diseño responsive
- Modo oscuro nativo
- Componentes reutilizables
- Utilidades personalizadas

Ejemplo de Uso:

```
<button class="px-4 py-2 bg-blue-600 text-white rounded-md hover:bg-blue-700  
    dark:bg-blue-700 dark:hover:bg-blue-800">  
    Guardar  
</button>
```

6.3 Sistema de Modales

Los modales utilizan JavaScript vanilla para su funcionamiento:

- Apertura/cierre con atributos `data-modal-open` y `data-modal-close`
- Gestión de estado con clases CSS
- Soporte para dark mode

6.4 Dark Mode

Implementación del modo oscuro:

```
// Toggle dark mode
document.documentElement.classList.toggle('dark');

// Guardar preferencia
localStorage.setItem('theme', isDark ? 'dark' : 'light');
```

7. Generación de PDFs y QR

7.1 DomPDF

Configuración para generar PDFs:

```
use Barryvdh\DomPDF\Facade\Pdf;

$pdf = Pdf::loadView('pdf.bien', ['bien' => $bien]);
$pdf->setPaper('a4', 'portrait');
return $pdf->download('bien_'.$bien->numero_bien.'.pdf');
```

7.2 Generación de QR

Uso de SimpleSoftwareIO/simple-qrcode:

```
use SimpleSoftwareIO\QrCode\Facades\QrCode;

$qqr = QrCode::size(200)
    ->format('png')
    ->generate(route('bienes-nacionales.show', $id));

return response($qqr->header('Content-Type', 'image/png'));
```

7.3 Plantillas PDF

Las plantillas PDF incluyen:

- Logo del hospital
- Información del bien/equipo
- Código QR embebido
- Estilos CSS inline

8. Instalación y Despliegue

8.1 Requisitos del Sistema

- PHP 8.2 o superior
- Composer 2.x
- MySQL 8.0 o superior
- Node.js 18.x o superior
- NPM 9.x o superior

8.2 Instalación Local

```
# Clonar repositorio
git clone [repository-url]
cd projectLaravel

# Instalar dependencias PHP
composer install

# Instalar dependencias Node
npm install

# Copiar archivo de entorno
cp .env.example .env

# Generar clave de aplicación
php artisan key:generate

# Configurar base de datos en .env
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=hospital_control
DB_USERNAME=root
DB_PASSWORD=

# Ejecutar migraciones
php artisan migrate

# Compilar assets
npm run build
```

```
# Iniciar servidor de desarrollo
php artisan serve
```

8.3 Configuración de Base de Datos

Crear la base de datos:

```
CREATE DATABASE hospital_control CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

8.4 Seeders (Opcional)

Para poblar la base de datos con datos de prueba:

```
php artisan db:seed
```

8.5 Despliegue en Producción

8.5.1 Optimizaciones

```
# Optimizar autoload
composer install --optimize-autoloader --no-dev

# Cachear configuración
php artisan config:cache

# Cachear rutas
php artisan route:cache

# Cachear vistas
php artisan view:cache

# Compilar assets para producción
npm run build
```

8.5.2 Configuración del Servidor

Configuración recomendada para Apache:

```
<VirtualHost *:80>
    ServerName hospital-control.local
    DocumentRoot /path/to/projectLaravel/public

    <Directory /path/to/projectLaravel/public>
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/hospital-error.log
    CustomLog ${APACHE_LOG_DIR}/hospital-access.log combined
</VirtualHost>
```

8.5.3 Permisos

```
# Dar permisos de escritura
chmod -R 775 storage
chmod -R 775 bootstrap/cache
chown -R www-data:www-data storage
chown -R www-data:www-data bootstrap/cache
```

8.6 Mantenimiento

8.6.1 Backups

Realizar backups regulares de:

- Base de datos
- Archivos subidos (`storage/app`)
- Archivo `.env`

8.6.2 Logs

Los logs se almacenan en `storage/logs/laravel.log`

8.6.3 Actualizaciones

```
# Actualizar dependencias
composer update
```

```
# Ejecutar nuevas migraciones
php artisan migrate

# Limpiar cachés
php artisan cache:clear
php artisan config:clear
php artisan route:clear
php artisan view:clear
```

8.7 Troubleshooting

Error 500

Verificar:

- Permisos de `storage/` y `bootstrap/cache/`
- Configuración de `.env`
- Logs en `storage/logs/`

Error de Base de Datos

Verificar:

- Credenciales en `.env`
- Servidor MySQL activo
- Base de datos creada

Assets no se cargan

Ejecutar:

```
npm run build
php artisan view:clear
```

Apéndices

A. Comandos Artisan Útiles

Comando	Descripción
php artisan list	Lista todos los comandos disponibles
php artisan migrate:status	Muestra el estado de las migraciones
php artisan route:list	Lista todas las rutas registradas
php artisan make:controller	Crea un nuevo controlador
php artisan make:model	Crea un nuevo modelo
php artisan make:migration	Crea una nueva migración
php artisan tinker	Abre REPL de Laravel

B. Variables de Entorno Importantes

Variable	Descripción
APP_NAME	Nombre de la aplicación
APP_ENV	Entorno (local/production)
APP_DEBUG	Modo debug (true/false)
APP_URL	URL base de la aplicación
DB_*	Configuración de base de datos

C. Contacto y Soporte

Para soporte técnico o consultas sobre el desarrollo:

- **Departamento de Sistemas**
- **Hospital Virgen Del Valle**

Versión del Manual: 1.0 - Enero 2026